

BONUS

LangGraph

Building agents as graphs, with state and loops

For agents that need to branch, loop, and remember

[AI/ML Engineer Learning Series · Deep Dive Edition](#)

What it is	A framework for building agents as graphs
Made by	The LangChain team
Core idea	Steps are nodes; state flows between them
Adds over LangChain	Loops, branching, full control
Best for	Complex agents that need to think in cycles

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	What is LangGraph?	Graphs instead of straight chains
2	Chains vs Graphs	Why loops and branches matter
3	The Core Concepts	State, nodes, edges
4	State — The Shared Memory	What flows through the graph
5	Building a Simple Graph	A working example
6	Conditional Edges	Letting the graph decide
7	Cycles — The Agent Loop	Looping back to think again
8	LangGraph vs LangChain vs CrewAI	Which to use when
9	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. What is LangGraph?

LangGraph is a framework, built by the LangChain team, for building AI agents as graphs instead of straight-line chains. Remember chains from the LangChain topic — step 1, then step 2, then step 3, always in order. Real agents often don't work that way: they need to loop back, try again, or take different paths depending on what happens. LangGraph is built exactly for that.

The name says it: instead of a chain (one straight line), you build a graph — a network of steps (nodes) connected by paths (edges), which can branch, merge, and loop. This gives you far more control over how an agent thinks and acts, which is why LangGraph has become a popular choice for building serious, production-grade agents.

■ *This is a natural next step after Agentic AI Concepts, PydanticAI, and CrewAI. LangGraph gives you the most control of all of them — useful once you understand the basic agent loop and want to build something more precise.*

2. Chains vs Graphs

A chain always flows one way: step 1 finishes, step 2 starts, and so on, straight through. A graph is more flexible: from any step, you can go to different next steps depending on a decision, and you can even go back to a step you've already visited.

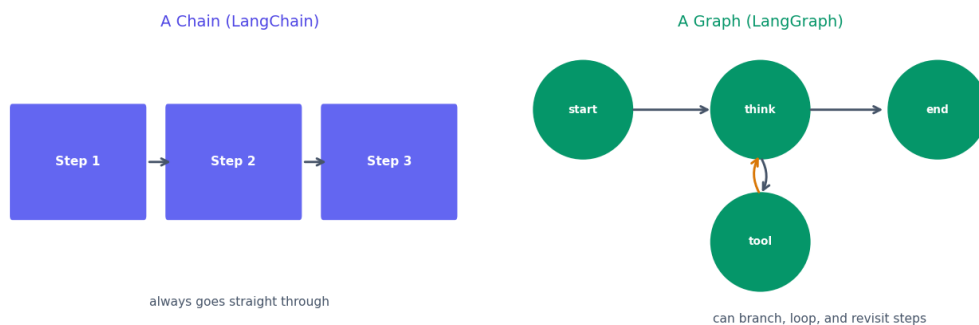


Fig 1 — A chain always goes straight through. A graph can branch and loop back to earlier steps.

Why does this matter for agents? Because real agent behaviour is often a loop: think, act, observe, then think AGAIN based on what happened — exactly the Think-Act-Observe loop from the Agentic AI topic. A straight chain can't easily represent 'go back and think again'. A graph can, naturally, with a loop.

3. The Core Concepts

LangGraph has three main ideas. Once you know these, the rest is just connecting them.

Concept	What it is
State	The shared data that flows through the whole graph
Node	A step — a function that does something and updates the state
Edge	A connection saying which node runs next

You define the state (what information matters), the nodes (the steps, like 'call the LLM' or 'use a tool'), and the edges (which step follows which). LangGraph then runs the graph, moving from node to node, updating the state along the way, until it reaches an end point.

4. State — The Shared Memory

State is the most important idea in LangGraph. It's a shared piece of data that every node can read and update as the graph runs — like a clipboard passed from person to person, where each person can read what's on it and add their own notes.

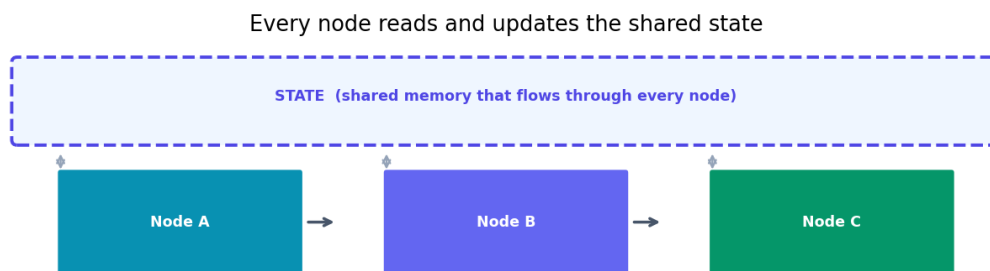


Fig 2 — State is shared memory: every node reads it, does its job, and updates it for the next node.

```
from typing import TypedDict

# Define the shape of your state (like a Pydantic model)
class AgentState(TypedDict):
    question: str
    search_results: str
    answer: str
```

This state travels through every node in the graph. A search node might fill in `search_results`; an answer node reads that and fills in `answer`. Because everything flows through one shared state, nodes stay simple — each just reads what it needs and writes what it produces.

5. Building a Simple Graph

Here's a small working graph: ask a question, search for info, then answer. Notice the pattern — define nodes as functions, then wire them together.

```
# pip install langgraph
from langgraph.graph import StateGraph, END

# 1. Define nodes (each is a function)
def search(state):
    results = search_web(state['question'])
    return {'search_results': results}

def answer(state):
    reply = llm(f"Answer using: {state['search_results']}")
    return {'answer': reply}

# 2. Build the graph
graph = StateGraph(AgentState)
graph.add_node('search', search)
graph.add_node('answer', answer)

# 3. Connect the nodes with edges
graph.set_entry_point('search')
graph.add_edge('search', 'answer')
graph.add_edge('answer', END)

# 4. Run it
app = graph.compile()
result = app.invoke({'question': 'What is RAG?'})
print(result['answer'])
```

■ Each node is just a normal Python function that takes the state and returns updates to it. `add_edge` says which node runs next. This simple search-then-answer graph is basically a tiny RAG pipeline, built the LangGraph way.

6. Conditional Edges

This is where graphs go beyond chains. A conditional edge lets the graph itself decide which node to go to next, based on the current state — like an if/else built into the flow.

Conditional Edge: the graph picks the path itself

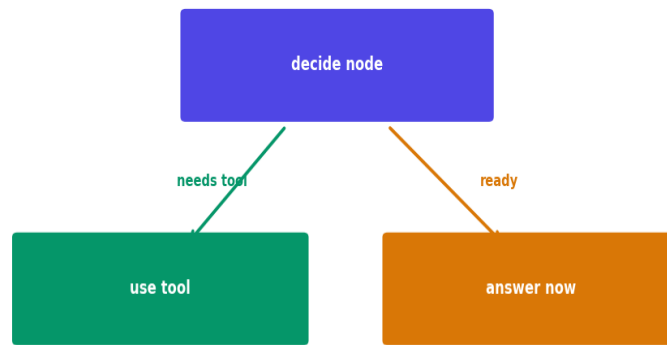


Fig 3 — A conditional edge: the graph checks the state and picks a different path.

```
def needs_tool(state):
    # returns the NAME of the next node
    if 'search' in state['question'].lower():
        return 'use_tool'
    return 'answer_now'

graph.add_conditional_edges(
    'decide',          # the node making the decision
    needs_tool,        # the function that decides
    {'use_tool': 'search', 'answer_now': 'answer'}
)
```

This is exactly how a smart agent decides 'do I need a tool for this, or can I answer directly?' — the decision is built into the graph structure itself, not just buried in one big prompt. It makes complex agent logic easy to see and reason about.

7. Cycles — The Agent Loop

The other superpower of graphs: they can loop. An edge can point BACK to an earlier node, creating a cycle. This is exactly the Think-Act-Observe loop from the Agentic AI topic, built directly into the graph structure.

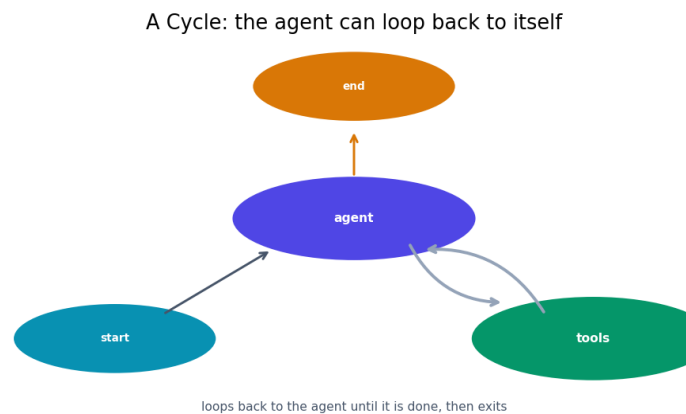


Fig 4 — A cycle: the agent can loop back to itself, repeating until it decides it's done.

```
# The agent node can loop back to itself via tools
graph.add_conditional_edges(
    'agent',
    should_continue,      # checks: keep going, or finish?
    {'continue': 'tools', 'finish': END}
)
graph.add_edge('tools', 'agent')  # loop back after using a tool!
```

This is the core pattern behind most modern LangGraph agents: the agent node thinks, may call a tool, the tool node runs and loops back to the agent, which thinks again with the new information — repeating until it decides to stop. A cycle is just an edge pointing backward.

■ Always cap the number of loops (LangGraph supports a recursion limit). Just like the safety habit from the Agentic AI topic, an uncapped cycle can loop forever if the agent gets confused. Set a max step count.

8. LangGraph vs LangChain vs CrewAI

You now know three agent-related tools. Here's how they compare, to help you pick the right one for a task.

	LangChain	CrewAI	LangGraph
Structure	Straight chains	Team of agents	Graph (branches + loops)
Control	Simple, less control	Role-based teamwork	Full, precise control
Best for	Simple pipelines	Multi-agent collaboration	Complex, looping agents
Learning curve	Easy	Easy	A bit steeper

Simple way to choose: LangChain for a simple, mostly-linear pipeline. CrewAI when you want a clear team of specialist agents. LangGraph when your agent's logic itself is complex — branching decisions, loops, retries — and you want to see and control that structure explicitly. Many advanced agents (including some behind well-known AI products) are built on LangGraph.

9. Common Mistakes

Mistake	Why it's a problem	Fix
No loop limit	Agent can loop forever	Set a recursion / step limit
Overcomplicating simple tasks	Graph is overkill	Use a plain chain if it's linear
Messy, huge state	Hard to track what's happening	Keep state small and clear
Forgetting <code>graph.compile()</code>	App won't run	Always compile before invoking
No clear END	Graph doesn't know when to stop	Route to END explicitly
Skipping the basics first	Confusing without agent grounding	Learn Topic 36 (Agentic AI) first

Quick Reference Cheatsheet

Concept / Task	Meaning or Code
LangGraph	build agents as graphs, not just chains
State	shared data flowing through the graph
Node	a step / function that updates state
Edge	connection saying what runs next
Conditional edge	the graph picks the path itself
Cycle	an edge that loops back — the agent loop
Install	<code>pip install langgraph</code>
Define state	<code>class X(TypedDict): ...</code>
Create graph	<code>StateGraph(X)</code>
Add a node	<code>graph.add_node('name', fn)</code>
Add an edge	<code>graph.add_edge('a', 'b')</code>
Conditional	<code>graph.add_conditional_edges(...)</code>
Run it	<code>graph.compile().invoke({...})</code>

The one idea to remember: LangGraph builds agents as graphs of nodes and edges, with a shared state flowing through them. Unlike a straight chain, a graph can branch with conditional edges and loop with cycles — giving you precise control over complex, real-world agent behaviour.

This is a bonus topic, extending your AI Agents stage beyond the original 44. It pairs well with what you already know from Agentic AI Concepts, PydanticAI, and CrewAI — a natural next step when you want more control over how your agents think.